

Technical Report – Assignment 02

Histogram Processing: Stretching, Equalization, and Matching

컴퓨터공학과 | 2317022 이다영

개요 (Overview)

본 과제에서는 OpenCV와 C++을 이용하여 이미지의 히스토그램을 분석하고 조작하는 프로그램을 구현하였다. 구현한 프로그램은 **PDF_CDF.cpp**, **hist_stretching.cpp**, **hist_eq.cpp**, **hist_eq_RGB.cpp**, **hist_eq_YUV.cpp**, **hist_matching_gray.cpp**, **hist_matching_color.cpp** 총 7개이며, 공용 유틸리티 함수는 **hist_func.h** 헤더 파일에 구현하였다.

모든 파일에서는 기존 코드의 deprecated API를 최신 OpenCV v4.x 호환 API로 수정하였다:

1. CV_LOAD_IMAGE_COLOR → IMREAD_COLOR
2. CV_RGB2GRAY → COLOR_BGR2GRAY
3. CV_RGB2YUV → COLOR_BGR2YUV

각 파일 빌드 후 생성되는 PDF/CDF 텍스트 문서는 파일로 별도로 제출한다. 본 보고서는 필요한 경우, 결과 이미지와 PDF/CDF 히스토그램 그래프를 첨부하였다.

hist_func.h

hist_func.h 는 PDF, CDF 계산 및 히스토그램 시각화에 사용할 공용 함수와 상수를 정의한다. 이 헤더를 포함하면 모든 .cpp 파일에서 동일한 함수를 재사용할 수 있다.

1. 기본 설정 및 타입 정의

```
#define L 256      // intensity level 수 (0 ~ 255)
#define IM_TYPE CV_8UC3
typedef uchar G;   // 8-bit grayscale 픽셀 타입
typedef cv::Vec3b C; // 8-bit 3채널 컬러 픽셀 타입
```

8비트 이미지에서 픽셀 intensity는 0~255 범위를 가지므로 L=256으로 설정한다. IM_TYPE 매크로를 통해 이미지 타입을 유연하게 변경할 수 있으며, 타입에 따라 G, C typedef가 자동으로 결정된다.

2. cal_PDF() — Grayscale PDF 계산

PDF(Probability Density Function)는 intensity r 을 가진 픽셀이 전체에서 차지하는 확률이다.

$$PDF(r) = \frac{n_r}{N}$$

여기서 n_r 은 intensity r 인 픽셀 수, N 은 전체 픽셀 수이다.

```
float* cal_PDF(Mat& input) {
    int count[L] = { 0 };
    float* PDF = (float*)calloc(L, sizeof(float));
    for (int i = 0; i < input.rows; i++)
        for (int j = 0; j < input.cols; j++)
            count[input.at<G>(i, j)]++;
    for (int i = 0; i < L; i++)
        PDF[i] = (float)count[i] / (float)(input.rows * input.cols);
    return PDF;
}
```

count 배열에 각 intensity의 빈도를 누적한 뒤 전체 픽셀 수로 나누어 확률 값(0.0~1.0)을 구한다. calloc으로 초기화된 배열을 동적 할당하며, 사용 후 호출측에서 반드시 free()해야 한다.

3. cal_CDF() — Grayscale CDF 계산

CDF(Cumulative Distribution Function)는 intensity가 r 이하인 픽셀의 누적 확률이다.

$$CDF(r) = \sum_{k=0}^r PDF(k) = \sum_{k=0}^r \frac{n_k}{N}$$

```
for (int i = 0; i < L; i++) {
    CDF[i] = (float)count[i] / (float)(input.rows * input.cols);
    if (i != 0)
        CDF[i] += CDF[i - 1];
}
```

$i=0$ 일 때는 이전 누적값이 없으므로 PDF[0] 값만 저장하고, $i \geq 1$ 일 때는 직전 CDF 값을 더해 누적을 유지한다. 마지막 인덱스 CDF[255]는 반드시 1.0이 된다.

4. cal_PDF_RGB() / cal_CDF_RGB() — 컬러 이미지

컬러 이미지는 B, G, R 3개의 채널을 가지므로, 각 채널에 대해 독립적으로 PDF와 CDF를 계산해야 한다. 반환 타입은 float** (256행 × 3열)이다.

```
// Count: 채널 k의 intensity i 빈도 누적
for (int k = 0; k < 3; k++)
    count[input.at<Vec3b>(i, j)[k]][k]++;

// PDF 계산: PDF[intensity][channel]
PDF[i][k] = (float)count[i][k] / (float)(input.rows * input.cols);

// CDF 계산: 채널별 독립 누적
CDF[i][k] = PDF[i][k];
if (i != 0) CDF[i][k] += CDF[i-1][k];
```

OpenCV에서 컬러 행렬의 (i, j) 픽셀 접근은 `mat.at<Vec3b>(i, j)[k]` 형태를 사용하며, $k=0$ 은 B, $k=1$ 은 G, $k=2$ 는 R 채널에 해당한다.

5. plot() / drawGraph() — 히스토그램 시각화

PDF 또는 CDF 데이터를 바탕으로 512×250 크기의 막대 그래프를 생성하여 별도의 창에 출력한다.

```
// x축: intensity 0~255 → 0~512 좌표 매핑
int pointX = margin + cvRound((i / 255.0) * x);
// y축: 값 → 높이 좌표 매핑 (위가 0이므로 y+margin에서 뺌)
int pointY = margin + y - cvRound((function[i] / maxSize) * y);
line(graph, Point(pointX, y+margin), Point(pointX, pointY), Scalar(255,255,255));
```

opt 파라미터로 PDF/CDF를 구분하며, PDF의 y축 최댓값은 기본 0.025(입력에 따라 동적 조정), CDF는 1.2로 설정한다. x축 라벨은 15 간격, PDF의 y축 라벨은 maxSize/6 간격, CDF는 0.2 간격으로 표시한다.

PDF_CDF.cpp

입력 이미지를 Grayscale로 변환한 뒤, PDF와 CDF를 계산하여 텍스트 파일과 그래프로 출력하는 가장 기본적인 프로그램이다.

1. 주요 처리 흐름

```
Mat input = imread("input.jpg", IMREAD_COLOR);
Mat input_gray;
cvtColor(input, input_gray, COLOR_RGB2GRAY); // BGR -> Grayscale

float* PDF = cal_PDF(input_gray); // PDF 계산
float* CDF = cal_CDF(input_gray); // CDF 계산

plot(PDF, "PDF", "PDF graph");
plot(CDF, "CDF", "CDF graph");

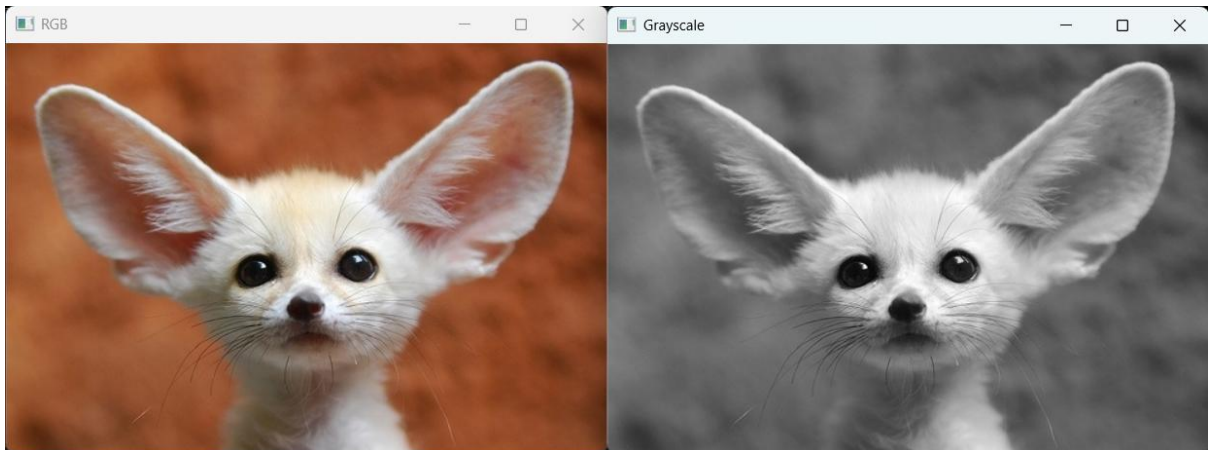
free(PDF); free(CDF); // 메모리 해제
```

imread()는 기본적으로 BGR 순서로 이미지를 읽어 온다. COLOR_RGB2GRAY 플래그를 이용해 Grayscale로 변환한 뒤 cal_PDF(), cal_CDF() 함수로 히스토그램 데이터를 추출한다.

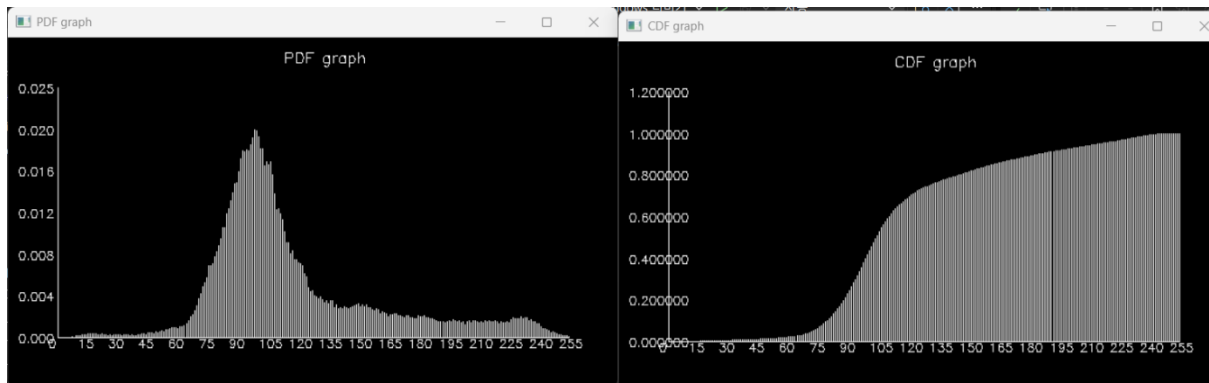
2. 실행 결과

아래는 input.jpg를 입력으로 사용하였을 때의 RGB/Grayscale 이미지와 PDF/CDF 그래프 출력 결과이다.

[RGB 원본 이미지 및 Grayscale 변환 결과]



[PDF graph (좌)/CDF graph (우)]



hist_stretching.cpp

히스토그램이 특정 intensity 구간에 집중된 low-contrast 이미지에 대해 Linear Stretching Function을 적용하여 히스토그램을 넓은 범위로 분산시키는 프로그램이다.

1. 이론 — Linear Stretching

구간별 서로 다른 기울기를 가진 선형 변환 함수를 적용한다. 일반 형식은 다음과 같다:

$$y = \left(\frac{(b_{i+1} - b_i)}{(a_{i+1} - a_i)} \right) (x - a_i) + b_i$$

x_1, x_2 를 경계로 intensity를 세 구간으로 나누어 각기 다른 변환 기울기를 적용한다. 중간 구간($x_1 \sim x_2$)의 기울기를 크게 설정함으로써 해당 구간의 히스토그램이 넓게 퍼지는 stretching 효과를 얻는다.

2. linear_stretching() 구현

```
void linear_stretching(Mat& input, Mat& stretched,
    G* trans_func, G x1, G x2, G y1, G y2) {
    float s1 = (float)y1 / x1;           // 구간1: 0 ~ x1
    float constant = (y2 - y1) / (float)(x2 - x1); // 구간2: x1 ~ x2
    float s3 = (float)(L-1-y2) / (L-1-x2); // 구간3: x2 ~ 255

    for (int i = 0; i < L; i++) {
        if (i <= x1)
            trans_func[i] = (G)(s1 * i);
        else if (i <= x2)
```

```

        trans_func[i] = (G)(constant * (i - x1) + y1);
    else
        trans_func[i] = (G)(s3 * (i - x2) + y2);
    }
    // 픽셀 매핑
    for (int i = 0; i < input.rows; i++)
        for (int j = 0; j < input.cols; j++)
            stretched.at<G>(i, j) = trans_func[input.at<G>(i, j)];
}

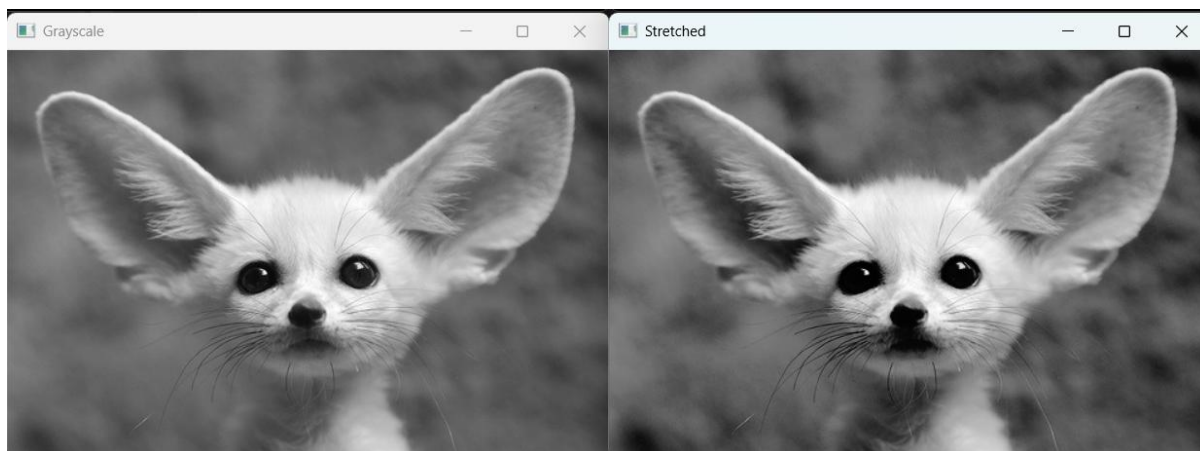
```

호출 예: `linear_stretching(input_gray, stretched, trans_func_stretch, 50, 110, 10, 110)`

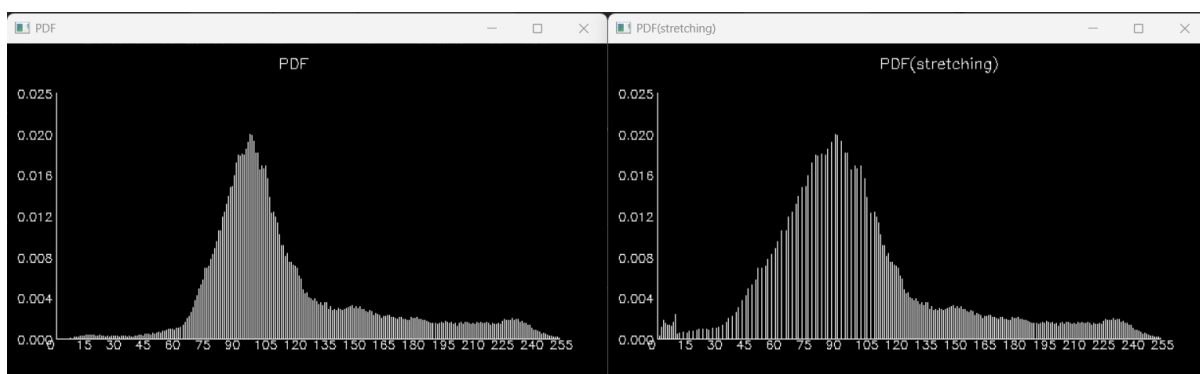
input.jpg의 PDF 분석 결과 intensity 50~110 구간에 픽셀이 집중되어 있으므로, 해당 구간을 (10, 110)으로 확장한다. 기존 분포가 더 넓은 범위에 걸쳐 나타나면서 명암비가 향상된다.

4. 실행 결과

[Grayscale 원본 (좌)/Stretched 결과 (우)]



[원본 PDF (좌)/Stretched PDF (우)]



hist_eq.cpp — Grayscale Histogram Equalization

Histogram Equalization은 이미지의 히스토그램을 최대한 균등하게 분포 시켜 contrast를 자동으로 개선하는 기법이다. 사용자 파라미터 없이 fully automatic하게 동작한다.

1. 이론 — Equalization Transfer Function

이상적인 equalization 결과는 모든 intensity의 PDF가 균등한(uniform) 상태이다. 이를 달성하는 transfer function은 CDF를 이용해 유도된다.

$$s = T(r) = (L - 1) * CDF_r(r)$$

증명 과정:

Step 1 변수 변환 공식 적용

$$PDF_s(s) = PDF_r(r) \cdot |dr/ds|$$

Step 2

$T(r) = (L - 1) \cdot CDF_r(r)$ 를 가정하고 ds/dr 계산

$$ds/dr = d/dr[(L - 1) \cdot CDF_r(r)] = (L - 1) \cdot d/dr CDF_r(r) = (L - 1) \cdot PDF_r(r)$$

Step 3

Step 1에 대입

$$PDF_s(s) = PDF_r(r) \cdot |dr/ds| = PDF_r(r) \cdot 1/[(L - 1) \cdot PDF_r(r)] = 1/(L - 1)$$

Step 4

$T(0) = 0$ 조건 하에 적분으로 확인

$$\int_0^r (L - 1) \cdot PDF_r(t) dt = (L - 1) \cdot CDF_r(r) = T(r) - T(0) = T(r)$$

Conclusion: Transfer function

$$s = T(r) = (L - 1) \cdot CDF_r(r)$$

2. hist_eq() 구현

```
void hist_eq(Mat& input, Mat& equalized, G* trans_func, float* CDF) {
```

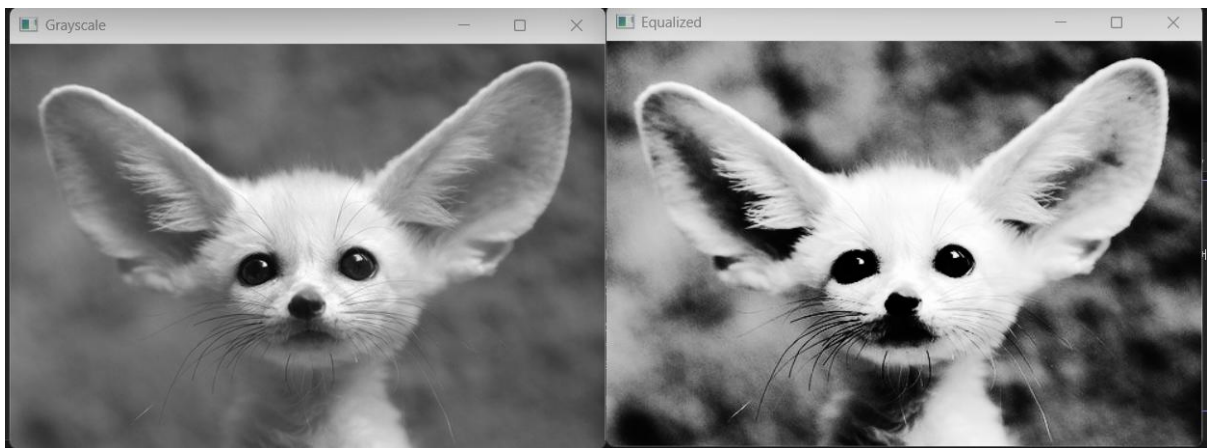
```
// Step 1: Transfer Function 계산
for (int i = 0; i < L; i++)
    trans_func[i] = (G)((L - 1) * CDF[i]);

// Step 2: 픽셀 단위 매핑
for (int i = 0; i < input.rows; i++)
    for (int j = 0; j < input.cols; j++)
        equalized.at<G>(i, j) = trans_func[input.at<G>(i, j)];
}
```

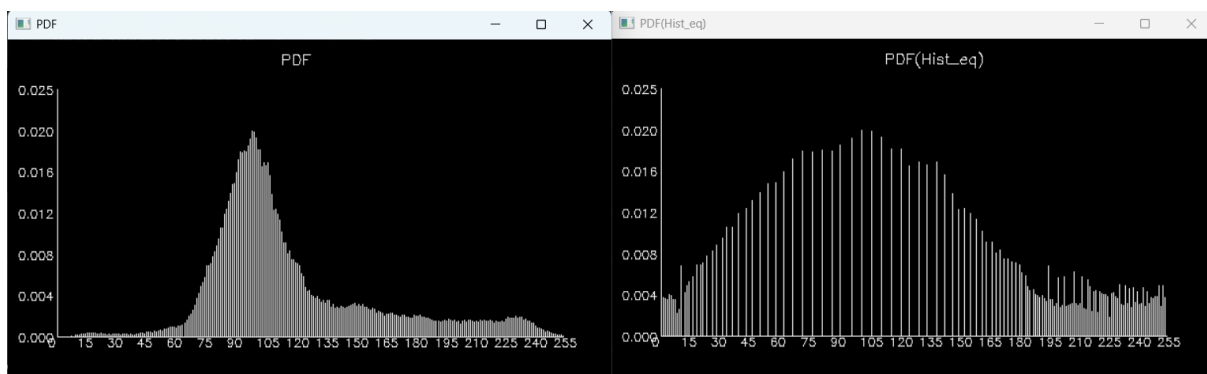
(G) 캐스트는 float 계산 결과를 0~255 범위의 8비트 정수로 자동 truncate한다. 모든 픽셀에 대해 원본 intensity를 인덱스로 사용해 변환된 intensity를 조회하는 Look-Up Table(LUT) 방식이므로 연산이 매우 효율적이다.

3. 실행 결과

[Grayscale 원본 (좌) / Equalized 결과 (우)]



[원본 PDF (좌) / Equalized PDF (우)]



4. Equalization 의 문제점

Histogram Equalization은 이미지 전체의 밝기 분포를 균일하게 재배치하는 과정에서, 원본 이미지의 밝은 영역(High-intensity)에 위치한 픽셀들을 출력 범위의 최댓값(255) 근처로 과도하게 밀집 시킨다. 이로 인해 밝은 부분의 세부 질감이 사라지고 하얗게 뭉쳐 보이는 washed-out 현상이 발생한다.

실제 이미지 분석(위 첨부된 이미지 참조)

- 귀 안쪽과 이마 부분: 원본 Grayscale 이미지에서는 사막여우의 귀 안쪽 털이나 이마의 미세한 결이 잘 표현되어 있으나, Equalization 결과물에서는 이 영역의 픽셀값들이 모두 255 에 가깝게 수렴하면서 털의 질감이 사라지고 평면적인 흰색으로 나타난다.
- 배경 및 노이즈: 이미지의 전반적인 명암비는 높아졌으나, 배경의 어두운 부분에 숨겨져 있던 미세한 잡음(Noise)까지 함께 증폭되어 시각적으로 거친 느낌을 주게 된다.

수치적/수학적 근거 (PDF 활용)

- PDF 분포 변화: 결과 이미지의 PDF 그래프를 보면, 밝기값이 전 구간에 걸쳐 넓게 퍼졌지만, 동시에 특정 고밝기 구간으로 픽셀들이 과도하게 쏠리면서 해당 구간의 변별력을 잃게 됨을 수치적으로 확인할 수 있다.

hist_eq_RGB.cpp — RGB 채널 독립 Equalization

3채널 컬러 이미지의 R, G, B 각 채널에 대해 독립적으로 Histogram Equalization을 적용하는 프로그램이다.

1. hist_eq_Color() 구현

```
void hist_eq_Color(Mat& input, Mat& equalized,
                  G(*trans_func)[3], float** CDF) {
    // 채널별 Transfer Function 계산
    for (int k = 0; k < 3; k++)
        for (int j = 0; j < L; j++)
            trans_func[j][k] = (G)((L - 1) * CDF[j][k]);
```

```

// 픽셀 단위 매핑
for (int i = 0; i < input.rows; i++)
    for (int j = 0; j < input.cols; j++)
        for (int k = 0; k < 3; k++)
            equalized.at<Vec3b>(i,j)[k]
                = trans_func[input.at<Vec3b>(i,j)[k]][k];
}

```

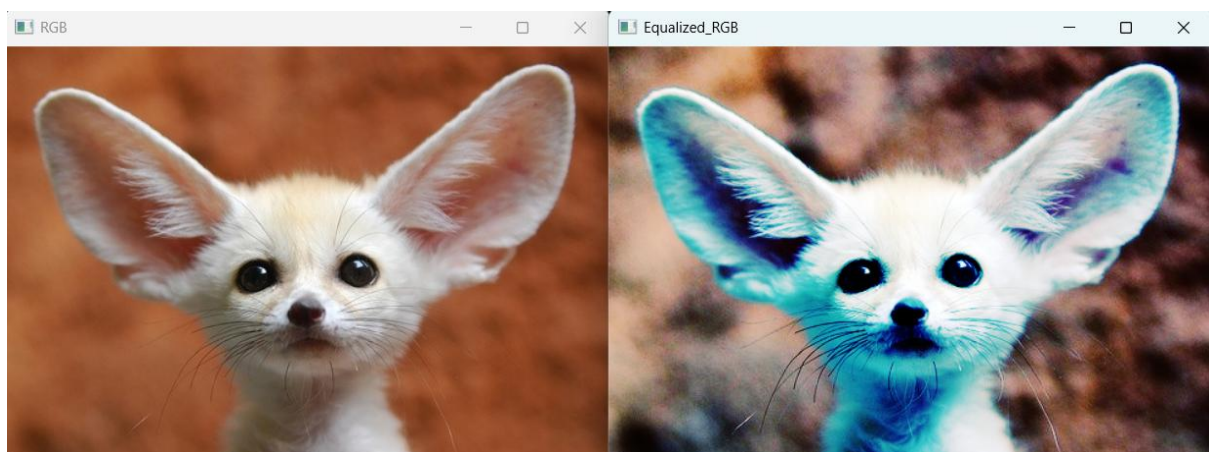
float** CDF는 [intensity][channel] 구조의 2D 배열이며, trans_func는 [intensity][channel] 구조의 2D 배열 포인터이다. 각 채널의 CDF를 이용해 채널별 변환 함수를 독립적으로 계산한다.

2. RGB 독립 연산의 문제점

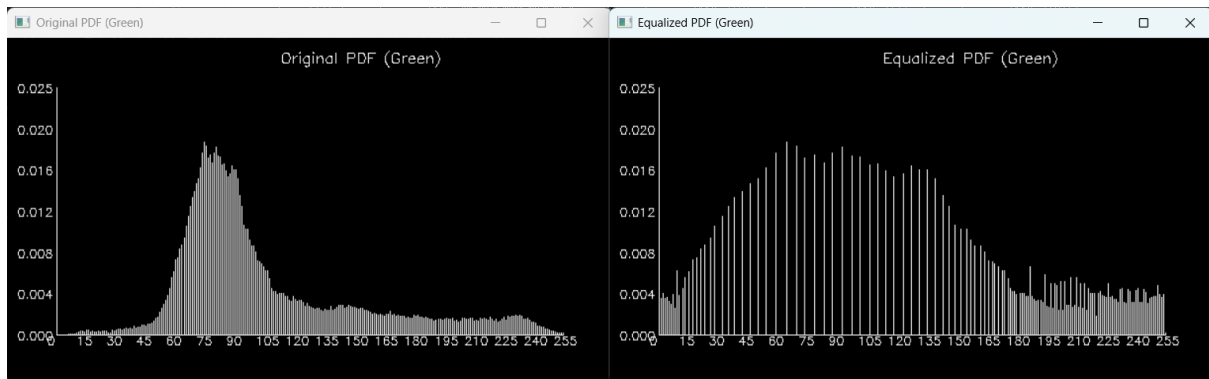
R, G, B 채널은 색상 정보를 공유하는 correlated 관계이다. 각 채널을 독립적으로 equalization하면 채널 간 상대적 비율이 달라져 색상 왜곡(color distortion)이 발생한다. 아래 결과 이미지와 같이 원본에 없던 색조가 나타날 수 있다.

3. 실행 결과

[RGB 원본 (좌) / RGB 독립 Equalization 결과 (우) — 색상 왜곡 확인]



[원본 PDF (좌) / Equalization 결과 PDF (우) --- green 채널을 대표로 출력]



hist_eq_YUV.cpp — YUV 공간 Equalization

RGB 채널 독립 연산의 색상 왜곡 문제를 YUV 색상 공간을 통해 해결한다. 명암 정보(Y)와 색상 정보(U, V)를 분리하여 Y 채널에만 equalization을 수행한다.

1. 처리 흐름

- ① BGR → YUV 변환(cvvtColor + COLOR_BGR2YUV)
- ② Y, U, V 채널 분리(split)
- ③ Y 채널에만 Histogram Equalization 적용
- ④ Y(equalized), U, V 채널 병합(merge)
- ⑤ YUV → BGR 역변환 (cvvtColor + COLOR_YUV2BGR)

2. 핵심 코드

```
// BGR -> YUV 변환
cvvtColor(input, equalized_YUV, COLOR_BGR2YUV);

// Y 채널 분리
Mat channels[3];
split(equalized_YUV, channels);
Mat Y = channels[0];

// Y 채널 equalization
hist_eq(Y, channels[0], trans_func_eq_YUV, CDF_YUV);

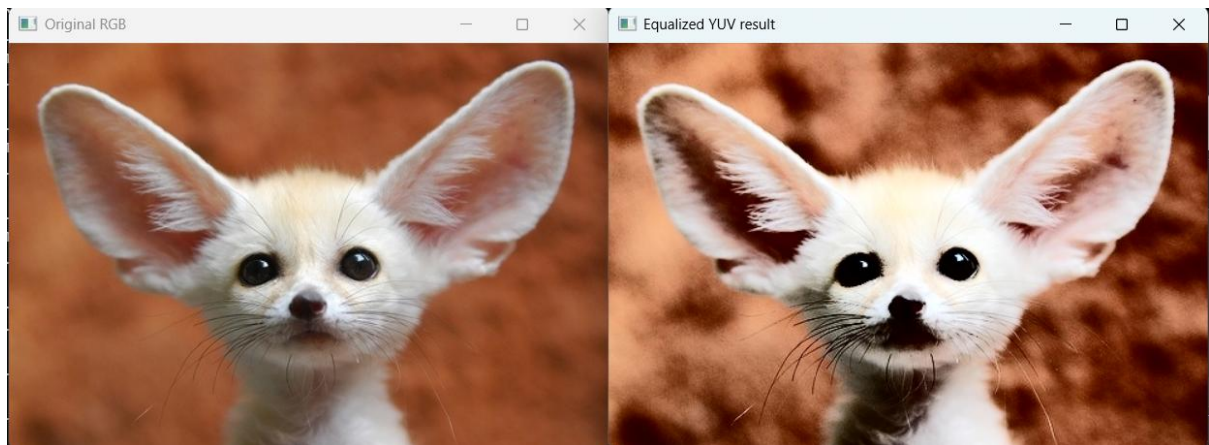
// 채널 병합 및 BGR 복원
```

```
merge(channels, 3, equalized_YUV);  
cvtColor(equalized_YUV, result_BGR, COLOR_YUV2BGR);
```

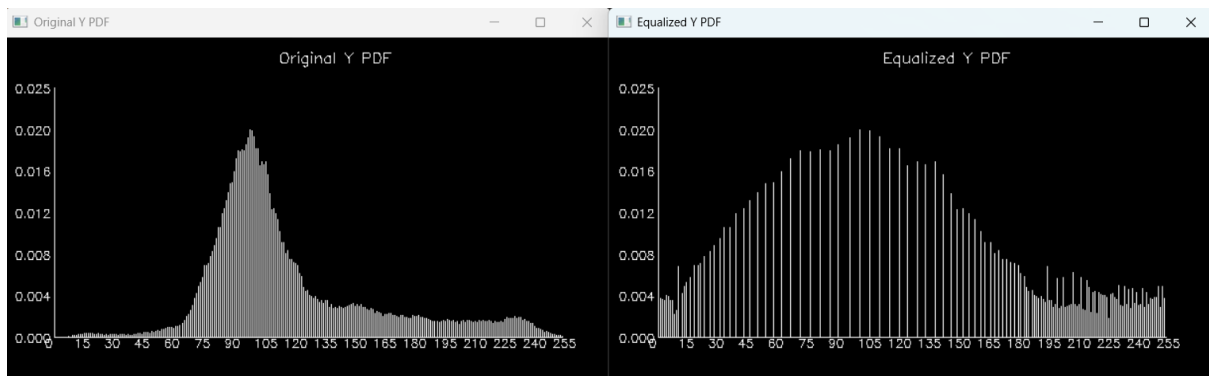
YUV 공간에서 Y(Luma)는 밝기만을 나타내며 U, V(Chroma)는 색상 정보를 담는다. Y 채널에만 equalization을 적용하면 색상 분포(U, V)는 그대로 유지되므로 색상 왜곡 없이 contrast만 개선된다.

3. 실행 결과

[RGB 원본 (좌) / YUV Equalization 결과 (우) — 색상 보존 확인]



[원본 PDF (좌) / YUV Equalization 결과 PDF (우)]



hist_matching_gray.cpp — Grayscale 히스토그램 매칭

입력 이미지의 히스토그램을 reference 이미지의 히스토그램 분포와 유사하게 만드는 기법이다. Stretching이나 Equalization과 달리 목표 분포를 reference 이미지로 지정할 수 있다. 구현 코드 내에서 두 이미지 파일의 이름은 input_image_flower.jpg 와 input.jpg이다.

```
// 이미지 로드
Mat input = imread("input_image_flower.jpg", IMREAD_COLOR);
Mat reference = imread("input.jpg", IMREAD_COLOR);
```

1. 이론 — Histogram Matching

매칭 알고리즘은 두 단계의 equalization을 이용해 $r \rightarrow z$ 매핑을 구성한다:

- 입력 이미지에 equalization 적용: $s = T(r) = (L - 1) * CDF_r(r)$
- Reference 이미지에 equalization 적용: $s = G(z) = (L - 1) * CDF_z(z)$
- 역함수를 이용해 $r \rightarrow z$ 매핑 구성: $z = G^{-1}(s) = G^{-1}(T(r))$

2. hist_matching() 구현

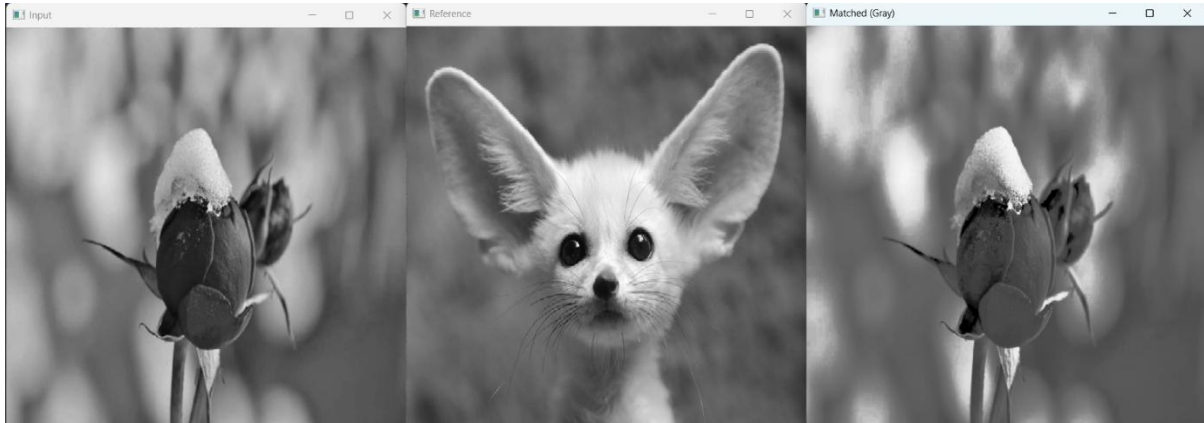
```
void hist_matching(Mat& input, Mat& matched,
                  G* trans_func_z, G* T, G* G_map) {
    // r에 대해 최적의 z 탐색 (최소 차이 방식)
    for (int r = 0; r < L; r++) {
        int s = T[r];          // T(r): 입력 equalization 값
        int best_z = 0;
        int min_diff = L;
        for (int z = 0; z < L; z++) {
            int diff = abs((int)G_map[z] - s);
            if (diff < min_diff) {
                min_diff = diff;
                best_z = z;
            }
            if (diff == 0) break; // 정확히 일치 시 조기 종료
        }
        trans_func_z[r] = (G)best_z;
    }
    // 픽셀 매핑
    for (int i = 0; i < input.rows; i++)
        for (int j = 0; j < input.cols; j++)
            matched.at<G>(i, j) = trans_func_z[input.at<G>(i, j)];
}
```

$G(z)$ 의 역함수 G^{-1} 는 일대일 대응이 아닐 수 있으므로, $G(z)$ 값 중 $T(r)$ 에 가장 가까운 z 를 선택하는 **최소 거리(nearest neighbor) 방식**을 사용한다. $\text{diff} == 0$ 인 경우 루프를 조기 종료

하여 연산 효율을 개선하였다.

3. 실행 결과

[Input (좌) / Reference (중) / Matched 결과 (우) — Grayscale]



hist_matching_color.cpp — Color 히스토그램 매칭

컬러 이미지에 대한 히스토그램 매칭 프로그램이다. hist_eq_YUV.cpp와 동일하게 YUV 공간을 활용하여 Y 채널에만 매칭을 적용함으로써 색상 왜곡을 방지한다.

hist_matching_gray.cpp 와 같이 이미지는 input_image_flower.jpg 와 input.jpg 두개를 이용했다.

```
// 이미지 로드
Mat input = imread("input_image_flower.jpg", IMREAD_COLOR);
Mat reference = imread("input.jpg", IMREAD_COLOR);
```

1. 처리 흐름

- ① 입력/reference 이미지 모두 BGR → YUV 변환
- ② 각 이미지의 Y 채널 분리 및 CDF 계산
- ③ Y 채널에 대해 Equalization Transfer Function $T(r)$, $G(z)$ 계산
- ④ hist_matching() 호출: Y 채널에만 매칭 적용 ($z = G^{-1}(T(r))$)
- ⑤ 채널 병합 후 YUV → BGR 역변환

2. 핵심 코드

```
// 두 이미지 모두 YUV 변환
```

```

cvtColor(input, equalized_input, COLOR_BGR2YUV);
cvtColor(reference, equalized_ref, COLOR_BGR2YUV);

// Y 채널의 Equalization Transfer Function 계산
hist_eq(Y_input, channels_input[0], trans_func_input, CDF_input);
hist_eq(Y_ref, channels_ref[0], trans_func_ref, CDF_ref);

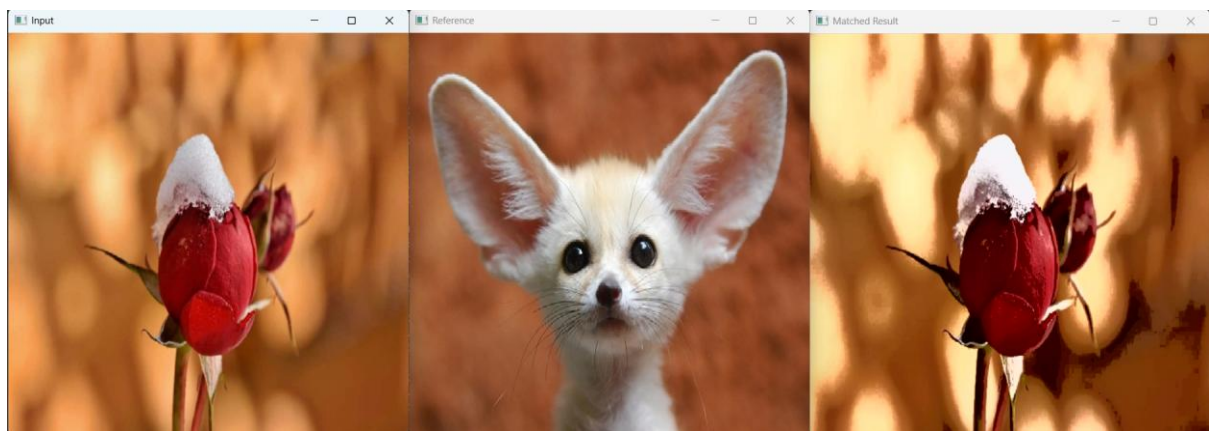
// Y 채널에만 Histogram Matching 수행
hist_matching(Y_input, channels_matched[0],
              trans_func_z, trans_func_input, trans_func_ref);

// 병합 후 BGR 복원
merge(channels_matched, 3, matched_YUV);
cvtColor(matched_YUV, matched_BGR, COLOR_YUV2BGR);

```

3. 실행 결과

[Input (좌) / Reference (중) / Matched 결과 (우) — Color]



결론

본 과제를 통해 이미지 히스토그램을 다루는 세 가지 핵심 기법(Stretching, Equalization, Matching)을 직접 구현하고 그 효과를 분석하였다.

- **Histogram Stretching:** 사용자가 직접 x_1 , x_2 , y_1 , y_2 파라미터를 지정해야 하지만, 특정 구간의 대비를 선택적으로 향상시킬 수 있다.

- **Histogram Equalization:** 파라미터 없이 자동으로 동작하는 fully automatic 방식이나, 밝은 이미지에서 washed-out 현상이 발생할 수 있다.
- **RGB vs YUV Equalization:** RGB 채널을 독립적으로 처리하면 색상 왜곡이 발생한다. YUV 공간의 Y 채널에만 적용하면 색상을 보존하면서 contrast 를 개선할 수 있다.
- **Histogram Matching:** reference 이미지의 분포를 목표로 입력 이미지를 변환한다. CDF 기반의 역함수 매핑을 통해 $r \rightarrow z$ 대응 관계를 구성한다.

배운점 및 시사점

본 과제를 통해 단순히 OpenCV 라이브러리 함수를 호출하는 수준을 넘어, 히스토그램이라는 데이터 분포가 이미지의 시각적 품질에 어떤 수학적 원리로 기여하는지 깊이 있게 이해할 수 있었다. 특히 PDF와 CDF를 직접 코드로 구현하며 이론적인 수식이 픽셀 단위의 연산으로 치환되는 과정을 경험한 것이 가장 큰 수확이었다. 또한, RGB 채널을 독립적으로 equalize 했을 때 발생하는 색상 왜곡 문제를 확인하고, YUV 색 공간의 Y(휘도) 채널만을 제어하여 원본의 색감을 보존하는 기법을 실습하며 데이터의 특성에 맞는 적절한 처리 방식의 중요성을 체득했다.

더불어, 히스토그램 평활화 과정에서 나타나는 washed-out 현상을 직접 분석해 보며, 모든 상황에 완벽한 단일 알고리즘은 없으며 데이터의 분포에 따라 최적의 전처리 기법을 선택해야 한다는 통찰을 얻을 수 있었다. 이번 경험은 향후 더 복잡한 컴퓨터 비전 및 데이터 분석 프로젝트를 수행할 때 밑거름이 될 수 있을 것 같다.